

Báo cáo rà soát test code

Lưu ý nghiêm trọng

- Không phát hiện.

Test còn thiếu

- TC-021: Kiểm tra deactivate() thành công khi số dư bằng 0 (được mô tả trong Test Plan nhưng chưa có trong mã test).
- Kiểm tra deposit và withdraw khi tài khoản đã bị deactivate (phải ném IllegalStateException từ checkActive()).
- Kiểm tra transfer khi số tiền lớn hơn số dư hiện tại của tài khoản nguồn (phải ném IllegalStateException do withdraw không đủ tiền).
- Kiểm tra transfer khi số tiền bằng 0 hoặc âm đã có, nhưng không có kiểm tra cho trường hợp số tiền > số dư.
- Kiểm tra isActive sau khi deactivate (được báo trạng thái chuyển thành false).
- Kiểm tra getBalance sau các thao tác deposit, withdraw, transfer để xác nhận không có thay đổi bất ngờ (được báo phụ lục kèm theo).

Assertion yếu

- Các test TC-018 và TC-019 xác nhận thông báo lỗi "Withdraw amount must be positive" của withdraw. Đây là chỉ thị nội bộ; nếu thông báo thay đổi, test sẽ thất bại dù hành vi vẫn đúng. Nên kiểm tra chính xác nội dung lỗi, không cần so sánh message, hoặc tạo message chung cho cả deposit/withdraw/transfer.
- Không có assertion kiểm tra trạng thái isActive sau deactivate() (thiếu trong TC-020 và TC-021).

Rủi ro maintainability

- Phụ thuộc vào thông báo ngoại lệ (message) khi test dễ bị break khi thay đổi văn bản.
- Lập lại tạo mới tồng new BankAccount(...) trong nhiều test; nếu thay đổi constructor hoặc giá trị mặc định, phải sửa nhiều chỗ.
- Không có @BeforeEach để khởi tạo các tài khoản dùng chung, làm tăng độ dài test và khó bảo trì.

Gợi ý sửa ngay

1. Thêm các test còn thiếu:

@Test

@DisplayName("TC-021: Deactivate succeeds when balance is zero")

```
void testDeactivateSucceedsWhenBalanceZero() {
```

```
    BankAccount account = new BankAccount("ACC011", 0.0);
```

```
    account.deactivate();
```

```
    assertFalse(account.isActive());
```

```
}
```

@Test

@DisplayName("Deposit throws IllegalStateException when account is inactive")

```
void testDepositWhenInactive() {
```

```
    BankAccount account = new BankAccount("ACC012", 0.0);
```

```
    account.deactivate();
```

```
    assertThrows(IllegalStateException.class, () -> account.deposit(10.0));
```

```
}
```

@Test

@DisplayName("Withdraw throws IllegalStateException when account is inactive")

```
void testWithdrawWhenInactive() {
```

```

BankAccount account = new BankAccount("ACC013", 10.0);
account.deactivate();
assertThrows(IllegalStateException.class, () -> account.withdraw(5.0));
}

```

@Test

@DisplayName("Transfer throws IllegalStateException when amount exceeds source balance")

void testTransferInsufficientFunds() {

BankAccount source = new BankAccount("SRC", 50.0);

BankAccount target = new BankAccount("TGT", 0.0);

assertThrows(IllegalStateException.class, () -> source.transfer(target, 100.0));

}

2. Rút gọn assertion và message:

- Thay assertEquals("Withdraw amount must be positive", exception.getMessage()); bằng assertTrue(exception.getMessage().contains("positive")); hoặc chỉ assertThrows(...); nếu không cần kiểm tra nội dung.

3. Sử dụng @BeforeEach để tạo các tài khoản chung:

```
private BankAccount account;
```

```
@BeforeEach
```

```
void setUp() {
```

```
account = new BankAccount("ACC000", 100.0);
```

```
}
```

4. Tách các hằng số (account numbers, số dư) thành static final để tránh lỗi typo và dễ thay đổi.

5. Kiểm tra isActive sau deactivate() trong test hiện tại (TC020) để chắc chắn trạng thái đã thay đổi khi không ném ngoại lệ.

Áp dụng các sửa đổi trên sẽ nâng cao bảo mật, giảm phụ thuộc vào chi tiết nội bộ và cải thiện khả năng bảo trì của test.